

UNIVERSIDAD NACIONAL DEL ALTIPLANO PUNO
FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA,
ELECTRÓNICA Y SISTEMAS
ESCUELA PROFESIONAL INGENIERÍA DE SISTEMAS



CURSO:

Programación Orientada a Objetos-II

TRABAJO:

Revisión de POO I: Diagnostico, Refactorización y Calidad de Código
(Parte 2)

PRESENTADO POR:

Melvin Ronald Choque Chaiña

DOCENTE:

Mg. Aldo Hernan Zanabria Galvez

FECHA:

03/09/2026

PUNO-PERÚ

Contenido

1. Problema y línea base	3
2. Olores de diseño identificados.....	3
3. Responsabilidades propuestas	4
4. Refactorizaciones aplicadas.....	5
5. Resultados de pruebas y métricas	5
6. Discusión: mejora lograda y costo introducido	6
7. Declaración de uso de IA.....	6
Referencias	7

1. Problema y línea base

El problema original (**gestor_academico_base.py** / **gestor_academico_base.cpp**) consiste en una solución monolítica para la gestión de notas académicas. En esta versión inicial, toda lógica de negocio, persistencia de datos, formateo de salida y trazabilidad se encuentra centralizada en una única entidad monolítica (**GestorAcademicoMonolitico**).

La línea base presenta severas deficiencias operativas y de mantenibilidad:

- Falta de integridad de datos: Acepta valores arbitrarios para calificaciones fuera del rango válido [0.0, 20.0].
- Acoplamiento rígido a I/O: Escribe directamente en archivos locales (**auditoria.txt** y **reporte.csv**), lo que impide la ejecución de pruebas unitarias aisladas en memoria.
- Fragilidad del estado: La colección interna de estudiantes se expone de manera pública, permitiendo mutaciones no controladas desde cualquier punto del programa.

2. Olores de diseño identificados

A través de la inspección estática del código base, se identificaron los siguientes code smells y violaciones a los principios SOLID:

Hallazgo	Ubicación	Riesgo	Refactorización candidata
Responsabilidades mezcladas	Métodos agregar_nota y reporte	Viola SRP. Cambios en la auditoria o el reporte pueden romper la lógica del dominio.	Extraer clases Auditoria , GeneradorReporte y ServicioAcademico
Estado expuesto	self.estudiantes (Python)/ estudiantes (C++)	Colección pública modificable	Ocultar atributo (_estudiante) y

		desde fuera sin control ni validaciones	exponer métodos de acceso controlado
Validación ausente	Método agregar_nota	Permite ingresar notas fuera del rango [0,20].	Trasladar la validación a la entidad Estudiante lanzando ValueError .
Dependencia concreta	Open("reporte.csv") y open("auditoria.txt")	Acoplamiento directo al sistema de archivos local. Dificulta las pruebas unitarias.	Injectar abstraídos de auditoría y repositorio.
Duplicación potencial	Lógica de promedio en reporte	Si se agrega un nuevo reporte se duplicará la regla del promedio y la aprobación.	Encapsular promedio y esta_aprobado dentro de Estudiante .

3. Responsabilidades propuestas

Para resolver el acoplamiento y la falta de cohesión, se descompuso la arquitectura en cuatro componentes con límites claros:

1. **Estudiantes** (Entidad de Dominio): Almacena el código, nombre y listado de notas. Es el único responsable de validar que las notas pertenezcan al intervalo [0.0, 20.0] y de calcular su promedio y condición de aprobación (≥ 10.5).
2. **RepositorioEstudiantes** (Persistencia en Memoria): Encargado de la colección de entidades. Permite guardar, recuperar y listar objetos **Estudiante** sin conocer la lógica de negocio.

3. **ServicioAcademico** (Caso de Uso/Orquestador): Coordina los flujos principales (matricular estudiante, asignar nota) invocando al repositorio y registrando las acciones en el servicio de auditoria.
4. **GeneradorReporteCSV** (Presentacion/Salida): Extrae los datos del repositorio y genera el formato delimitado por comas (.csv) sin afectar las entidades de dominio.

4. Refactorizaciones aplicadas

Las refactorizaciones aplicadas se centran en dos principios fundamentales de diseño:

- Principios de Responsabilidad Única (SRP): Se fragmentó la superclase original en entidades independientes. Ninguna clase maneja simultáneamente procesamiento de datos y estructura en disco.
- Inversión de Dependencias (DIP): Se definió la interfaz/protocolo **AuditoriaInterface** (**IAuditoriaService** en C++). Servicio Academico depende exclusivamente de esta abstracción, permitiendo inyectar una implementación real (**AuditoriaArchivo** / **AuditoriaConsola**) o una nula (**AuditoriaNull**) durante la ejecución de pruebas unitarias.

5. Resultados de pruebas y métricas

Indicador	Antes	Después	Interpretación
Clases/Archivos afectados	1 clase (GestorAcademico Monolítico)	5 clases (Estudiante, RepositorioEstudiantes, ServicioAcademico, Generador ReporteCSV, AuditoriaArchivo/Null)	Se descompuso el monolito en componentes especializados con responsabilidad única (SRP).
Dependencias de Gestor/Servicios	Concretas (acoplamiento directo a disco mediante open/ofstream)	Abstractas (Inyección de interfaz AuditoriaInterface/IAuditoriaService)	Aplicación del principio DIP, permitiendo aislar la lógica de

			negocio de los detalles de I/O.
Métodos de la clase principal	3 métodos que hacían de todo (agregar_estudiante, agregar_nota, generar_reporte)	2 métodos coordinadores (registrar_estudiante, agregar_nota_estudiante)	Reducción de la complejidad ciclómata en el servicio orquestador.
Pruebas aprobadas / tota	0 / 0 (no existían pruebas automatizadas sin I/O)	4 / 4(100% pasadas en pytest)	Cobertura total sobre reglas de negocio (rango de notas, promedios y reportes) sin tocar disco.
Tiempo del cambio (min)	~ 45 min (código frágil y propenso a errores al modificar)	~ 10 min (cambios seguros y localizados en la clase correspondiente)	Alta mantenibilidad; modificar la auditoría o la exportación no afecta las reglas de dominio.

6. Discusión: mejora lograda y costo introducido

- Mejoras Logradas: Se logro una arquitectura extensible y modular. Ahora es posible cambiar el mecanismo de auditoria sin modificar la lógica académica. Asimismo, el sistema es completamente testeable de forma aislada.
- Costos Introducidos: La solución refactorizada incrementó la cantidad de clases y archivos en el proyecto, agregando un ligero nivel de abstracción inicial que se justifica plenamente en el escalar la aplicación.

7. Declaración de uso de IA

Se utilizó la herramienta Gemini (Google AI) como asistente interactivo durante el desarrollo para:

1. El análisis estático de olores de código y propuesta de la tabla de diagnóstico

2. La estructuración de las pruebas unitarias con pytest y la verificación del estándar C++20.
8. La maquetación y redacción del informe técnico conforme a la 14 de la guía de laboratorio.

Referencias

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Python Software Foundation. (2026). Python Documentation: typing.Protocol and dataclasses. <https://docs.python.org/3/>

ISO/IEC. (2020). Programming Languages — C++ (ISO/IEC 14882:2020). Standard for C++20.